VISITOR PATTERN — ADVANCED FILE SYSTEM

Name: Smat Zhangir

Group: SE-2522

Course: ShP-2216 | Software Design Patterns

Format: Individual | Java 17 | Maven

1. Introduction

The Visitor pattern is a behavioral design pattern that allows new operations to be added without modifying existing element classes. The purpose of this project is to implement this pattern using an in-memory hierarchical file system in Java 17. The system represents files and directories as objects and supports three operations: calculating total size, searching for files, and collecting statistics.

2. Design and Implementation

The file system uses the Element interface, an abstract Node class, and two concrete classes: FileNode and DirectoryNode. FileNode stores a filename and its byte size, while DirectoryNode contains child elements and supports nested directories.

Double dispatch is implemented through the accept() method. First, TreeWalker calls node.accept(visitor), selecting the correct element implementation. Then, the element calls visitor.visit(this), selecting the corresponding overloaded visit method.

TreeWalker handles preorder traversal and visits each reachable element exactly once. The root starts at depth 0, and child paths are relative to it, using / as the separator. Search results are sorted lexicographically.

The three visitors perform independent operations. TotalSizeVisitor calculates the sum of file sizes, SearchVisitor finds files matching a filename condition and an inclusive minimum-size threshold, and StatisticsVisitor counts files, directories, and maximum depth.

3. Clean Code Practices

Five Clean Code practices were applied in the project.

Meaningful naming: Classes and methods such as TotalSizeVisitor and TreeWalker.walkNode() clearly describe their purposes.

Single Responsibility: Each visitor performs one independent operation.

Small methods: FileNode.accept() and DirectoryNode.accept() contain only one visitor call.

Encapsulation: DirectoryNode.addChild() validates changes, while getChildren() returns an unmodifiable collection.

No type-checking chains: The overloaded Visitor.visit() methods eliminate the need for instance of checks and type-based branching.

4. Testing and Results

The project uses JUnit 5 and Maven for automated testing. FileSystemTest contains 10 tests covering double dispatch, nested size calculations, search conditions, deterministic paths, statistics, empty structures, repeated execution, and invalid tree mutations.

Test result: 10 tests passed, 0 failures (use this only if it matches your actual test run).

The program produces the following output:

Total bytes: 590

Java files >= 100 bytes: [src/App.java]

Files: 4

Directories: 3

Maximum depth: 2

5. Design Decisions and Trade-offs

TreeWalker handles traversal so that visitors only perform their specific operations. It resets visitors before each execution, preventing old results from affecting new calculations.

The system prevents negative file sizes, duplicate sibling names, multiple parents, and cycles. It also supports arbitrary nesting and empty directories.

StatisticsVisitor was added without modifying FileNode or DirectoryNode, demonstrating extensibility. However, adding a new element type would require updating the Visitor interface and every existing visitor implementation.

6. Conclusion

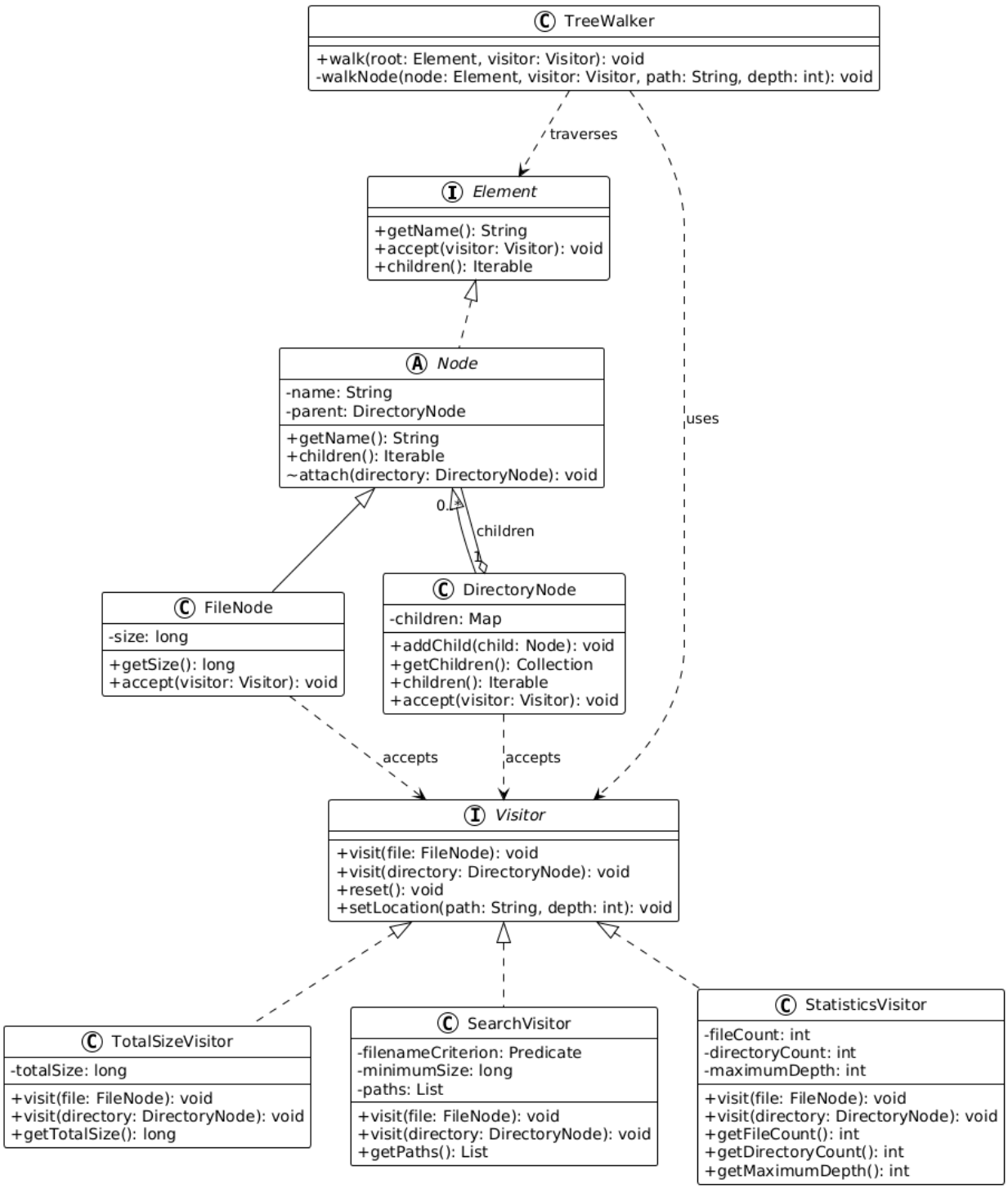
The project demonstrates how the Visitor pattern separates operations from an object structure through double dispatch. The implementation supports multiple independent operations, maintains tree integrity, and produces deterministic results verified through automated testing.

8. Repository and References

Repository: https://github.com/smatD/assignment-6-visitor-pattern

- Refactoring.Guru, Visitor: https://refactoring.guru/design-patterns/visitor
- Refactoring.Guru, Double Dispatch: https://refactoring.guru/design-patterns/visitor-double-dispatch
- Apache Maven, Java target: https://maven.apache.org/guides/mini/guide-java-target.html
- JUnit 5 User Guide: https://docs.junit.org/5.11.4/user-guide/

9. UML Class Diagram


TreeWalker
+walk(root: Element, visitor: Visitor): void
-walkNode(node: Element, visitor: Visitor, path: String, depth: int): void
traverses
Element
+getName(): String
+accept(visitor: Visitor): void
+children(): Iterable
Node
-name: String
-parent: DirectoryNode
+getName(): String
+children(): Iterable
~attach(directory: DirectoryNode): void
uses
0..*
children
1
FileNode
-size: long
+getSize(): long
+accept(visitor: Visitor): void
DirectoryNode
-children: Map
+addChild(child: Node): void
+getChildren(): Collection
+children(): Iterable
+accept(visitor: Visitor): void
accepts
accepts
Visitor
+visit(file: FileNode): void
+visit(directory: DirectoryNode): void
+reset(): void
+setLocation(path: String, depth: int): void
TotalSizeVisitor
-totalSize: long
+visit(file: FileNode): void
+visit(directory: DirectoryNode): void
+getTotalSize(): long
SearchVisitor
-filenameCriterion: Predicate
-minimumSize: long
-paths: List
+visit(file: FileNode): void
+visit(directory: DirectoryNode): void
+getPaths(): List
StatisticsVisitor
-fileCount: int
-directoryCount: int
-maximumDepth: int
+visit(file: FileNode): void
+visit(directory: DirectoryNode): void
+getFileCount(): int
+getDirectoryCount(): int
+getMaximumDepth(): int